STUDENT NAME: AMANDEEP	MODULE: My Practice	CODE: TEST_Q001_FACT
BATCH: ABES_JAS26_FSD JAVA_IIIYear_Batch 02	COURSE: Software Testing Debugging Concepts using AI	STATUS: In Progress

Practice Worksheet – Q001 – Calculate Factorial

Design exhaustive unit test suites for factorial algorithms: recursive vs iterative complexity, boundary value analysis, integer overflow detection, and AI-driven automated test generation.

KEY LEARNING OBJECTIVES:

- Implement Boundary Value Analysis (BVA) for integer factorial calculations.
- Detect and prevent arithmetic overflow in long data types (n > 20).
- Assert proper validation handling for negative inputs with IllegalArgumentException.
- Engineer AI prompts to synthesize parameterized edge-case unit test matrices.

STEP-BY-STEP WORKSHEET INSTRUCTIONS:

Step 1: Analyze the factorial algorithm specification and identify mathematical domain limits.
Step 2: Formulate test cases for equivalence classes: zero, unit, standard integers, and overflow boundaries.
Step 3: Write JUnit 5 test methods utilizing assertAll(), assertEquals(), and assertThrows().
Step 4: Verify complete test coverage across iterative and recursive implementation variants.

EXERCISE 1.1: MATHEMATICAL DOMAIN & EQUIVALENCE PARTITIONING

Problem Statement: Define the equivalence partitions for a method `long factorial(int n)`. Partition the domain into: Valid Small Integers (0 <= n <= 10), Valid Large Integers (11 <= n <= 20), Extreme Overflow Range (n >= 21), and Invalid Domain (n < 0). Specify expected outputs or exceptions for each partition class.

Reference Baseline / Scenario / Code:
factorial(0) -> 1, factorial(1) -> 1, factorial(5) -> 120, factorial(20) -> 2432902008176640000L

STUDENT SOLUTION & CODE WORKSPACE:
[1] Equivalence Partition Matrix: _____
[2] Boundary Values (Min, Nominal, Max): _____
[3] Expected Return / Exception: _____

EXERCISE 1.2: JUNIT 5 BOUNDARY TEST IMPLEMENTATION

Problem Statement: Write a complete JUnit 5 test method `testFactorialBoundaries()` asserting that `factorial(0) == 1`, `factorial(1) == 1`, `factorial(20) == 2432902008176640000L`, and that `factorial(21)` either throws `ArithmeticException` or wraps into `BigInteger`.

Reference Baseline / Scenario / Code:

```
@Test void testFactorialBoundaries() { assertEquals(1L, MathUtils.factorial(0)); }
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] JUnit 5 Test Annotations: _____

[2] Assertion Statements (assertAll): _____

[3] Boundary Value Verification: _____

EXERCISE 1.3: NEGATIVE INPUT & EXCEPTION HANDLING TEST

Problem Statement: Construct a test method `testNegativeInputThrowsException()` using `assertThrows()` to verify that passing negative integers (n = -1, -5, -100) immediately raises `IllegalArgumentException` with the specific message 'Input must be non-negative'.

Reference Baseline / Scenario / Code:

```
IllegalArgumentException ex = assertThrows(IllegalArgumentException.class, () -> MathUtils.factorial(-5));
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] assertThrows Syntax & Lambda: _____

[2] Error Message String Assertion: _____

[3] Edge Case Parameter Verification: _____

EXERCISE 1.4: AI PROMPT ENGINEERING FOR AUTOMATED TEST GENERATION

Problem Statement: Craft a comprehensive prompt instructing an AI coding assistant to generate a parameterized JUnit 5 test suite for `BigInteger factorial(BigInteger n)`. The prompt must mandate testing numbers up to 100, execution time benchmarks (< 50ms), and memory heap sanity constraints.

Reference Baseline / Scenario / Code:

```
Act as a Senior QA Automation Architect. Generate a parameterized JUnit 5 test suite for BigInteger factorial...
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] AI Prompt Persona & Directives: _____

[2] Test Requirements & Framework Rules: _____

[3] Negative Constraints & Output Schema: _____

SELF-REVIEW & VERIFICATION CHECKLIST:

- [ ] **Boundary Coverage:** Values n=0, 1, 20, 21 explicitly verified.
- [ ] **Exception Assertions:** Negative inputs assert IllegalArgumentException.
- [ ] **Overflow Prevention:** Detects and prevents 64-bit long precision truncation.
- [ ] **AI Prompt Clarity:** Prompt produces production-ready, compilable JUnit 5 tests.
- [ ] **Test Determinism:** Tests run in isolation with zero external state dependency.

CRITICAL REFLECTION & LEARNING SYNTHESIS:

- *Why does long overflow occur at n=21, and how can unit tests reliably catch silent wraparounds?*
- *How can AI test generation be guided to include edge cases that developers typically overlook?*

Reflection Notes:

_____

_____

COMPLETION STATUS:

[ ] COMPLETED (Meets All Solution Criteria)

[ ] NEEDS REVISION (Resubmit Corrected Work)

Evaluator Signature: _____

EVALUATOR & MENTOR REMARKS:

Feedback Notes:

_____

Rubric Evaluation: [ ] Correctness [ ] Code Quality [ ] Edge Cases